Problema del controllo ottimo applicato all'atterraggio del lander Philae sulla cometa 67P/Churyumov-Gerasimenko

Andrea Bianchini

Giugno 2025

1 Obiettivo dello Studio

Lo scopo di questo documento è illustrare come sono riuscito a trovare una traiettoria ottima per il rilascio del lander Philae sulla superficie della cometa 67P/Churyumov-Gerasimenko da parte della sonda Rosetta.

2 Breve Storia della Missione

La missione Rosetta dell'ESA è di fondamentale importanza storicamente perchè è la prima ad aver mandato una sonda in orbita intorno ad una cometa e di conseguenza anche la prima ad aver fatto atterrare un lander su di una cometa.

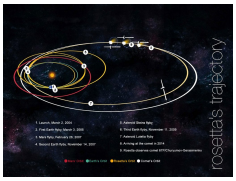


Figure 1: La Traiettoria di Rosetta

La spedizione è stata fondamentale per ampliare la nostra conoscenza sull'origine del sistema solare, dato che gran parte delle comete, essendo intrappolate nella fascia di Kuiper o nell'ipotetica nube di Oort, hanno subito pochissime alterazioni nel tempo. Inoltre la presenza di acqua e composti organici costituiscono un altro forte motivo d'interesse per gli scienziati. La sonda partì dal Centre Spatial Guyanais a bordo di un Ariane 5 il 2 Marzo 2004. Rosetta doveva entrare in un'orbita molto lenta intorno alla cometa, e successivamente rallentare la sua orbita fino ad arrestarla in modo tale da potersi preparare per il rilascio del lander Philae, diretto sulla superficie della cometa stessa.

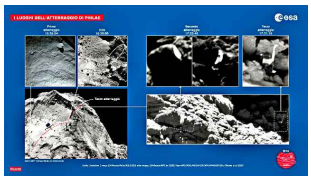


Figure 2: atterraggio di Philae

L'atterraggio sarebbe dovuto avvenire in una zona precisa della cometa, denominata Agilkia, individuata dall'occhio principale della sonda : VIRTIS. Agilkia fu selezionato come sito ideale, perchè si tratta di una zona particolarmente liscia e piatta della cometa, perfetta sia per permettere una discesa in sicurezza, senza il rischio che il lander possa incontrare qualche ostacolo, sia perché l'assenza di ostacoli permette una buona illuminazione (indispensabile per alimentare i pannelli solari di Philae, garantendo operazioni oltre la durata delle batterie primarie). Inoltre la zona sembrava mostrare anche una particolare varietà geologica e morfologica, ideale per le analisi che avrebbe dovuto condurre Philae. Il lander è stato sganciato dalla sonda il 12 novembre 2014 ad una velocità iniziale precisa, proseguendo in caduta libera verso la cometa, guidato dalla sua debole attrazione gravitazionale. Dopo circa 7 ore il lander raggiunse la superficie a circa 1,1 m/s, senza l'uso di retrorazzi (a causa di un malfunzionamento), in seguito il carrello di atterraggio,

dotato di di 2 arpioni, trivelle a vite e un piccollo motore a gas compresso avrebbero dovuto contribuire all'ancoraggio del lander alla superficie, impedendo eventuali rimbalzi, che data la bassa gravità potevano far spazzar via il lander dalla cometa. Tuttavia, probabilmente per un guasto al sistema di sparo, i due arpioni non si attivarono e il lander rimbalzò sulla superficie, rimanendo in aria per due ore prima di rimbalzare una seconda volta e terminare su una zona ombrosa della cometa (Abydos), motivo per cui dopo circa due ore di attività il lander si spense ed iniziò la fase di ibernazione.

3 Caratteristiche della Cometa

La cometa 67P/Churyumov–Gerasimenko è un piccolo corpo celeste con una forma irregolare, spesso descritta come simile a una papera di gomma, composta da due lobi uniti da un "collo" sottile. Ha un diametro medio di circa 4 chilometri e una massa molto bassa, con una gravità superficiale quasi nulla. La sua superficie è scura, coperta da uno strato di polveri e materiali organici, e mostra falesie, fratture, pianure e cavità profonde. La cometa è composta principalmente da una miscela di ghiaccio d'acqua, anidride carbonica, metano e polvere rocciosa, custodita al suo interno fin dalla formazione del Sistema Solare. Gli scienziati ritengono che 67P provenga dalla fascia di Kuiper — una regione ai margini del Sistema Solare ricca di oggetti ghiacciati — e che, a causa di interazioni gravitazionali con i pianeti giganti, abbia intrapreso un'orbita molto più interna. Ora appartiene alla famiglia delle comete gioviane di corto periodo, che orbitano attorno al Sole in meno di 20 anni.

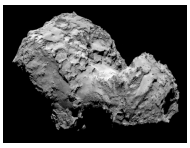


Figure 3: Cometa 67P/Churyumov–Gerasimenko. (da Rosetta)

4 Caratterisitche del lander Philae

Philae è un piccolo lander robotico sviluppato dall'Agenzia Spaziale Europea (ESA) e da varie agenzie partner, tra cui l'ASI, progettato per atterrare sulla superficie di una cometa e condurre esperimenti scientifici in situ. Il lander misura circa 1 metro cubo (1 × 1 × 0,8 metri) e ha una massa di 98 kg. È dotato di un piccolo propulsore a gas freddo e di tre gambe ammortizzate per l'atterraggio, ciascuna con viti per ancorarsi al suolo, e un sistema di arpioni esplosivi. L'alimentazione è fornita da batterie interne ricaricabili tramite pannelli solari. A bordo di Philae c'erano 10 strumenti scien-

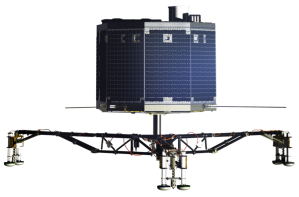


Figure 4: Lander Philae

tifici, tra cui una trivella (SD2) per prelevare campioni del suolo, un microscopio (ÇIVA), uno spettrometro per l'analisi chimica (COSAC), e sensori per misurare temperatura, campo magnetico e resistenza meccanica del terreno. Il lander comunicava con la sonda madre Rosetta, che fungeva da ponte verso la Terra.

5 Impostazione del Problema

5.1 Ipotesi Semplificative

- **Ipotesi di Cometa piatta**: è un'ipotesi molto forte data la forma marcatamente irregolare della cometa, ma è necessaria per poter assumere costante il campo gravitazionale e dunque per poter considerare g costante e diretta verso il basso (evitando componenti laterali)
- **Ipotesi di Cometa stazionaria rispetto al lander in discesa**: Anche questa è un'ipotesi abbastanza forte dato che la cometa ruota intorno al proprio asse con un periodo di 12.4 ore, inoltre la sua forma irregolare complica ulteriormente le cose, dato che il suo momento d'inerzia varia e può portare a fenomeni di precessione e nutazione.
- **Ipotesi di discesa puramente verticale**: semplifica notevolmente il modello matematico, evitando l'introduzione di componenti di velocità laterali.

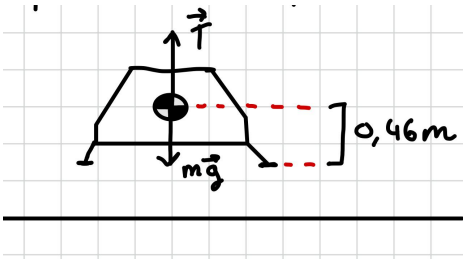
5.2 Condizioni Iniziali e Finali

Condizioni Iniziali	Condizioni Finali
$x_1^0 = 22500\text{m}$	$x_1^f = 0.46\text{m}$
$x_2^0 = -0.187\text{m/s}$	$x_2^f = 0$
$t_0 = 0$	$t_f = ?$

Queste condizioni iniziali e finali sono dovute al fatto che il lander è stato spinto da Rosetta, da un'altezza di 22.5km dalla superficie, con una velocità iniziale verso la cometa di 0.187m/s. La quota finale deve essere 0.46m perchè questa è la distanza tra baricentro del lander e zampe d'appoggio, e nelle equazioni dello stato considero il moto del baricentro di Philae.

5.3 Parametri Utili

- $m_0 = 98$ kg (massa iniziale di Philae)
- $T_{max} = 1\text{N}$ (spinta massima del motore)
- $g_{cometa} = 0.0001\ \text{m}/s^2$ (accelerazione gravitazionale della cometa)
- Isp = 60 (Impulso specifico del motore)



- c = $g_{terrestre} \cdot \text{Isp} = 588.6$ m/s (velocità di scarico)

nota: non sono riuscito a trovare il valore reale della spinta massima del propulsore, ma dato che il lander ha una massa relativamente bassa e che il valore dell'attrazione gravitazionale della cometa è anch'esso molto piccolo, ho considerato una spinta piuttosto bassa (1N).

5.4 Modello Matematico ed Equazioni dello Stato

Il moto del lander può essere schematizzato come in figura: Le equazioni che ne descrivono il moto sono:

$$\begin{cases} \dot{x}_1 = x_2 \\ \dot{x}_2 = \dfrac{T_{\max} u}{x_3} - g_{\text{cometa}} \\ \dot{x}_3 = -\dfrac{T_{\max}}{c} \end{cases}$$

6 Sviluppo del Problema di Ottimizzazione

Ora non ci resta che impostare il problema di controllo ottimo sulla traiettoria. Il mio obiettivo è quello di massimizzare la massa finale, e dunque quello di consumare meno carburante possibile. Dunque devo trovare un controllo u, che inserito nelle equazioni dello stato minimizza il mio indice di costo J e al contempo soddisfa i vincoli sullo stato ed al contorno (che in questo caso sono assenti). Per questo problema il controllo u è associato alla spinta massima del motore.

$$J = x_3^f$$

Si tratta di un indice di costo alla Mayer dato che quest'ultimo è definito come J = J(x_0,x_f,t_0,t_f). Per minimizzare il mio indice di costo devo trovarmi in un suo punto estremale, definito come un punto in cui il segno della variazione è costante. La condizione da imporre su J affinchè questo sia verificato è quella di stazionarietà : $\delta J = 0$. Da cui si ottengono le due equazioni di Eulero-Lagrange per il controllo ottimo, che devono essere sempre verificate lungo il moto per

garantire l'estremalità:

$$\begin{cases} \dot{P} - \dfrac{\partial H}{\partial x} = 0 \\ \dfrac{\partial H}{\partial u} = 0 \end{cases}$$

La prima ci permette di trovare la variazione nel tempo dei moltiplicatori P (co-stato) e la seconda ci permetterà di trovare il controllo u. Bisogna inoltre considerare le condizioni di trasversalità che nascono laddove una o più condizioni sullo stato iniziale o finale vengono lasciate libere. Le equazioni sono:

$$\begin{cases} P_0 = -\frac{\delta J}{\delta x_o} - \lambda \frac{\delta w}{\delta x_0} \\ H_0 = \frac{\delta J}{\delta t_o} + \lambda \frac{\delta w}{\delta t_o} \\ Pf = \frac{\delta J}{\delta x_f} + \lambda \frac{\delta w}{\delta x_f} \\ H_f = -\frac{\delta J}{\delta t_f} - \lambda \frac{\delta w}{\delta t_f} \end{cases}$$

Nel nostro caso le variabili libere sono il tempo finale e la massa finale, quindi sfruttando la terza e la quarta equazione, e ricordando che $J = x_3$ e che non ho vincoli al contorno $\rightarrow w = 0$, le condizioni di trasversalità sono:

$$\begin{cases} P_3^f = 1 \\ H^f = 0 \end{cases}$$

L'operazione successiva è definire la funzione Hamiltoniana "H". Questa per definizione è:

$$H = \sum_{i=1}^{N} P_i \dot{x}_i$$

dove N indica il numero di variabili di stato e gli P sono i moltiplicatori di Lagrange associati a ciascuna variabile di stato. In questo caso diviene :

$$\mathcal{H} = P_1 x_2 - P_2 \left(\frac{T_{\max} u}{x_3} - g_{\text{cometa}} \right) - P_3 \frac{T_{max} u}{c}$$

Ora siamo in grado di ricavare le equazioni del co-stato, che sono:

$$\begin{cases} \dot{P}_1 = -\frac{\delta H}{\delta x_1} = 0 \\ \dot{P}_2 = -\frac{\delta H}{\delta x_2} = -P_1 \\ \dot{x}_3 = -\frac{\delta H}{\delta x_3} = \frac{P_2 T_{max} u}{x_3^2} \end{cases}$$

Ora scrivo H dividendolo in una parte dipendente dal controllo + una parte indipendente dal controllo. Va fatto perchè questo problema di controllo ottimo fa parte di un particolare tipo di OCP detto "Problema Di Pontraygin". E si dimostra che per questo particolare tipo di problema, per massimizzare J, è sufficiente sostituire la seconda equazione di Eulero-Lagrange con

$$u^* = argsupH$$

con "u" appartenente al dominio di ricerca del vettore di controllo. Questo corrisponde all' applicazione del principio del massimo di Pontraygin, che richiede di massimizzare punto per punto l'Hamiltoniana rispetto al controllo. Isolare la parte di H dipendente da u ci permette di controllare la variabile u in modo che la condizione su H sia sempre verificata.

$$H = P_1 x_2 - P_2 g_{cometa} + T_{max} u[\frac{P_2}{x_3} - \frac{P_3}{c}]$$

da cui possiamo definire la funzione di switch $\Theta = [\frac{P_2}{x_3} - \frac{P_3}{c}]$ La funzione di switch è fondamentale, perchè ci permette di definire un controllo del tipo "on/off" sulla spinta del motore, infatti imponendo:

$$\begin{cases} \Theta > 0 \rightarrow u = 1 \\ \Theta < 0 \rightarrow u = 0 \\ \Theta = 0 \rightarrow u = ? \end{cases}$$

Deduciamo che la spinta del motore può solo essere massima o nulla. L'andamento di Θ è fondamentale dunque perchè ci indica quando la spinta del motore sarà attiva e quando sarà spenta. Quando $\Theta = 0$ in generale non abbiamo informazioni su u. Tuttavia in questo caso specifico si possono fare delle considerazioni analizzando la forma di Θ :

Se $\Theta = 0$ per un certo intervallo di t, anche $\dot{\Theta} = 0$ per quell'intervallo di t $\rightarrow \dot{\Theta} = -\frac{P_1}{x_3} = 0 \rightarrow P_1 = 0$. Se P_1=0 , la seconda equazione del co-stato ci dice che $\dot{P_2} = 0 \rightarrow P_2 = costante$, e poichè $H^f = 0$ per questo problema, $P_2^f = 0$ e dunque $P_2 = 0$ sempre. Se ora quindi pongo $\Theta = 0, P_1 = 0, P_2 = 0, P_3 = 1 \rightarrow \Theta = -\frac{1}{c}$ Il risultato pero è in contraddizione con l'ipotesi $\Theta = 0$, dunque questo indica che non ci possono essere archi singolari ma solo archi balistici o di spinta. Un'altra importante condizione da applicare, è quella sull'andamento dell'Hamiltoniana. Infatti se (come in questo caso) il t_f è libero, il valore ottimale di J può essere sensibile a variazioni infinitesime del tempo finale. Per impedire che ciò avvenga, la derivata della Hamiltoniana rispetto al tempo dev'essere nulla lungo la traiettoria ottima.

$$\frac{d\mathcal{H}}{dt} = 0$$

$$\frac{d\mathcal{H}}{dt} = \frac{\partial \mathcal{H}}{\partial t} + \frac{\partial \mathcal{H}}{\partial x}\dot{x} + \frac{\partial \mathcal{H}}{\partial u}\dot{u} + \frac{\partial \mathcal{H}}{\partial P}\dot{P}$$

Sostituendo le equazioni di stato e co-stato:

$$\dot{x} = \frac{\partial \mathcal{H}}{\partial p}, \quad \dot{P} = -\frac{\partial \mathcal{H}}{\partial x}$$

Otteniamo:

$$\frac{d\mathcal{H}}{dt} = \frac{\partial \mathcal{H}}{\partial t} + \frac{\partial \mathcal{H}}{\partial x}\frac{\partial \mathcal{H}}{\partial P} + \frac{\partial \mathcal{H}}{\partial P}\left(-\frac{\partial \mathcal{H}}{\partial x}\right) = \frac{\partial \mathcal{H}}{\partial t}$$

Quindi, se $\mathcal{H}$ non dipende esplicitamente dal tempo:

$$\frac{d\mathcal{H}}{dt} = 0 \quad \Rightarrow \quad \mathcal{H}(t) = \text{costante}$$

Ora dunque integrando le equazioni del co-stato, che con le giuste c.i., otterrò l'andamento di "$u(t)$" che se messo nello stato (anch'esso integrato con le rispettive c.i.) rende minimo l'indice di costo J.

7 Fmin

Per risolvere il problema di controllo ottimo ho utilizzato una funzione di nome "fmin", implementata in Matlab per trovare i valori ottimali degli P iniziali e del t_f tali da minimizzare la funzione di costo J. Questo è necessario perchè i valori iniziali degli P, vengono scelti arbitrariamente e dunque risulta difficile selezionarli in modo tale da trovare gli andamenti di P che minimizzano J. Fmin dunque si occupa di calcolare J partendo dagli P iniziali scelti arbitrariamente, cambiando questi valori e ricalcolando J fino a convergere verso un suo minimo locale. Per utilizzarla, dobbiamo introdurre una funzione di costo specifica da assegnare a fmin, che dipende dalle condizioni finali a cui vogliamo arrivare, che in questo caso sono:

- $x_1^f = 0.46$
- $x_2^f = 0$

  e quindi la funzione di costo da dare a fmin diventa:

$$J = \beta_1(x_1^f - 0.46) + \beta_2 x_2^f + \beta_3(P_3 - 1) + \beta_4 H_f$$

Ogni volta che fmin cambia i valori delle P iniziali, calcola J. Se J è alto fmin sa che deve re-iterare. Sappiamo di aver trovato la traiettoria ottima solo quando lo J è zero. Questo significa che le nostre condizioni sono soddisfatte. Tuttavia fmin è un'ottimizzatore locale, dunque riesce solamente a trovare il minimo locale di J che è più vicino alle P iniziali scelte da noi arbitrariamente, quindi non sempre riusciamo a trovare un J relativamente prossimo allo 0. Qui entrano in gioco gli algoritmi evoluzionistici come il Particle Swarm Optimizator, che ci permettono di individuare la zona di J in cui si trova il minimo globale, e dunque la miglior soluzione possibile.

7.1 Codice

L'ambiente dove ho utilizzato fmin è Matlab, di conseguenza ho scritto un codice che mi ha permesso di risolvere il problema di ottimizzazione diviso in tre script diversi.

7.1.1 Primo Script

```
% Script principale: ottimizzazione del lander Philae

clc; clear;
x1_f = 0.46;            % quota finale desiderata [m]
x2_f = 0;              % velocità finale desiderata [m/s]

% Stima iniziale per p1, p2, p3, tf
P_0 = [ -0.00056932, -11, 0.05, 19465.9349];

% Ottimizzazione
options = optimset('Display','iter');
P_opt = fminsearch(@lander_Philae, P_0, options);

% Parametri
m_0 = 98;
T_max = 1;
g_0 = 9.81;
g_l = 0.0001;
Isp = 60;
c = g_0 * Isp;

% Condizioni iniziali
x1_0 = 22500;
x2_0 = -0.187;
x3_0 = m_0;
X_0 = [x1_0 x2_0 x3_0];
I = [X_0 P_0];

% Simulazione ottimale
[t, X] = ode113(@(t,X) lander(t,X,T_max,g_l,c), [0 I(7)], I(1:6));

% Estrazione variabili
x1 = X(:,1);
x2 = X(:,2);
x3 = X(:,3);
p2 = X(:,5);
p3 = X(:,6);
```

```
theta = p2 ./ x3 - p3 / c;
u = double(theta > 0);
T = T_max * u;

% Plot dei risultati
figure(1)
subplot(2,2,1);
plot(t, x1);
xlabel('Tempo [s]'); ylabel('Quota [m]');
title('Quota');

subplot(2,2,2);
plot(t, x2);
xlabel('Tempo [s]'); ylabel('Velocità [m/s]');
title('Velocità');

subplot(2,2,3);
plot(t, x3);
xlabel('Tempo [s]'); ylabel('Massa [kg]');
title('Massa');

subplot(2,2,4);
plot(t, T);
```

7.1.2 Secondo Script

```
% funzione costo da minimizzare

function J = lander_Philae(P_0)
    % P_0 = [p1_0, p2_0, p3_0, tf]

    % Parametri fisici
    m_0 = 98;           % massa iniziale [kg]
    T_max = 1;         % spinta massima [N]
    g_0 = 9.81;            % gravità terrestre [m/s^2]
    g_l =  0.0001;            % gravità cometa [m/s^2]
    Isp = 60;             % impulso specifico [s]
    c = g_0 * Isp;

    % Condizioni iniziali
    x1_0 = 22500;           % quota iniziale [m]
    x2_0 = -0.187;            % velocità iniziale [m/s]
    x3_0 = m_0;            % massa iniziale
    X_0 = [x1_0 x2_0 x3_0];
    I = [X_0 P_0];         % stato iniziale completo: x1, x2, x3, p1, p2, p3, tf

    % Obiettivi finali
    x1_f = 0.46;
    x2_f = 0;



    % Simulazione dinamica
    [~, X] = ode113(@(t,X) lander(t,X,T_max,g_l,c), [0 I(7)], I(1:6));

    % Estrazione variabili
    x3 = X(:,3);
    p2 = X(:,5);
    p3 = X(:,6);

    % Controllo
    theta = p2 ./ x3 - p3 / c;
    u = double(theta > 0);
    % Hamiltoniana
    H = X(:,4) .* X(:,2) + X(:,5) .* (T_max * u ./ X(:,3) - g_l) - X(:,6) .* (T_max * u / c);
```

```
    H = X(:,4) .* X(:,2) + X(:,5) .* (T_max * u ./ X(:,3) - g_l) - X(:,6) .* (T_max * u / c);

    % Funzione costo (criterio di ottimalità)
   c1 = 1; c2 = 1; c3 = 0.001; c4 = 0.001;
   J = c1*abs(X(end,1) - x1_f) + ...
        c2*abs(X(end,2) - x2_f) + ...
        c3*abs(X(end,6) - 1) + ...
        c4*abs(H(end));
end
```

7.1.3 Terzo Script

```
%equazioni dello stato e del costato integrate da ode113

function f = lander(~, X, T_max, g_l, c)
    f = zeros(6,1);
    theta = X(5)/X(3) - X(6)/c;
    u = double(theta > 0);
    f(1) = X(2);
    f(2) = T_max * u / X(3) - g_l;
    f(3) = -T_max * u / c;
    f(4) = 0;
    f(5) = -X(4);
    f(6) = (X(5) * T_max * u) / (X(3)^2);
end
```

7.2 Ricerca delle costanti β e del vettore P_0 ideale

Le costanti β moltiplicano i vari termini della funzione di costo da dare a fmin e dunque sono in grado di conferire più o meno rilevanza al fatto che la condizione finale venga soddisfatta sul corrispettivo termine. In questo caso ho conferito più rilevanza alla condizione sulla quota finale e sulla velocità finale, dando meno importanza allaecondizioni su H_f e $P_3 \rightarrow c1 = 1e6, c_2 = 1e6, c3 = 1, c4 = 1$. La ricerca dei valori iniziali delle P da inserire in fmin è stata la parte più complessa della risoluzione del problema. Tuttavia nel processo ho sviluppato dei criteri che mi hanno portato a restringere notevolmente il campo di ricerca ed avvicinarmi nella zona dove ho pensato che si potesse trovare la soluzione desiderata. La prima cosa che ho notato era l'importanza fondamentale del tempo finale. infatti se questo era troppo basso,

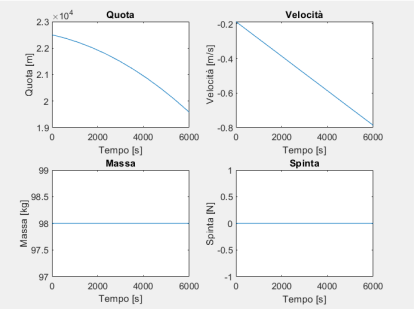


Figure 5: Run del programma con $t_f = 6000$

come si vede in figura 5, il lander non riesce a scendere più di una certa quota. Invece se il t_f viene imposto eccessivamente grande...

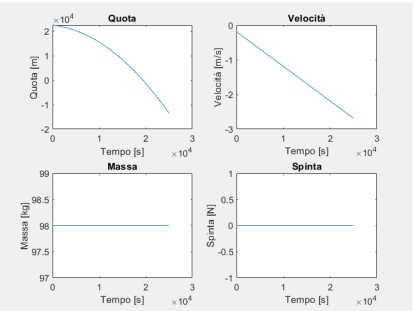


Figure 6: Run del programma con $t_f = 25000$

si vede che chiaramente supera la quota desiderata, non riuscendo ad attivare la spinta ed urtando in modo violento contro la cometa. Dunque ho pensato che potesse essere utile calcolare una stima del tempo che potrebbe aver impiegato il lander a scendere sulla superficie, includendo anche una fase finale di spinta che avrebbe rallentato la discesa. Considerando il fatto che sicuramente la soluzione da noi cercata è una per la quale il motore si accende nella fase finale della discesa, ho calcolato che se il lander si vuole fermare a 0.46m e che arriva negli ultimi 100m della discesa a circa 2.1m/s deve iniziare a spingere a circa 22.75m dalla superficie.

- $$v = \sqrt{2gh}$$

$$v = \sqrt{2 \cdot 0.0001 \cdot (22500 - 100)}$$

$$v \approx \sqrt{4.499908} \approx 2.121\,\mathrm{m/s}$$

- $$a = \frac{F_{\text{motore}}}{m} - g$$

$$a = \frac{1}{98} - 0.0001$$

$$a \approx 0.0102 - 0.0001 = 0.01\,\mathrm{m/s^2}$$

- $$d = \frac{v^2}{2a}$$

$$d = \frac{(2.121)^2}{2 \cdot 0.01}$$

$$d = \frac{4.5}{0.02} = 225\,\mathrm{m}$$

- calcolo del tempo di discesa una volta accesi i motori:

$$v = v_0 + at \implies t = \frac{v - v_0}{a} = \frac{0 - 2.121}{-0.01} = \frac{-2.121}{-0.01} = 212.1\,\mathrm{s}$$

Invece per il calcolo del tempo che il lander impiega per arrivare fino a 22.75m, ho considerato il grafico della quota ottenuto dalle simulazioni del programma, e ho approssimato l'ultimo tratto ad una retta, poi prendendo due punti su questo tratto, ho trovato l'equazione di questa retta, da cui ho ricavato approssimatamente il tempo impiegato dal lander per arrivare a 22.75m.

- Punto 1: $(x_1, y_1) = (18758.9, 1397.21)$
- Punto 2: $(x_2, y_2) = (20900.2, -3249.32)$
- $y = 22.75$ (valore dato per cui calcolare x)

1.pendenza della retta m:

$$m = \frac{y_2 - y_1}{x_2 - x_1} = \frac{-3249.32 - 1397.21}{20900.2 - 18758.9} = \frac{-4646.53}{2141.3} \approx -2.17$$

2.equazione della retta $y = mx + q$:
Usiamo il primo punto per trovare q:

$1397.21 = -2.17 \cdot 18758.9 + q \Rightarrow q = 1397.21 + 2.17 \cdot 18758.9 \Rightarrow q \approx 1397.21 + 40785.81 = 42183.02$

Quindi:

$$y = -2.17x + 42183.02$$

3. x **per cui** $y = 22.75$**:**

$$22.75 = -2.17x + 42183.02 \Rightarrow -2.17x = 22.75 - 42183.02 = -42160.27 \Rightarrow x = \frac{-42160.27}{-2.17} \approx 19432.1$$

Quindi t_f è approssimativamente $19432.1 + 212 = 19644.1$s . Per i valori degli P invece, provando vari valori iniziali casualmente, ho notato che la spinta spesso non si attivava, oppure quando si attivava, ciò accadeva solo durante la fase iniziale della missione, con il risultato che il lander iniziava a salire di quota.

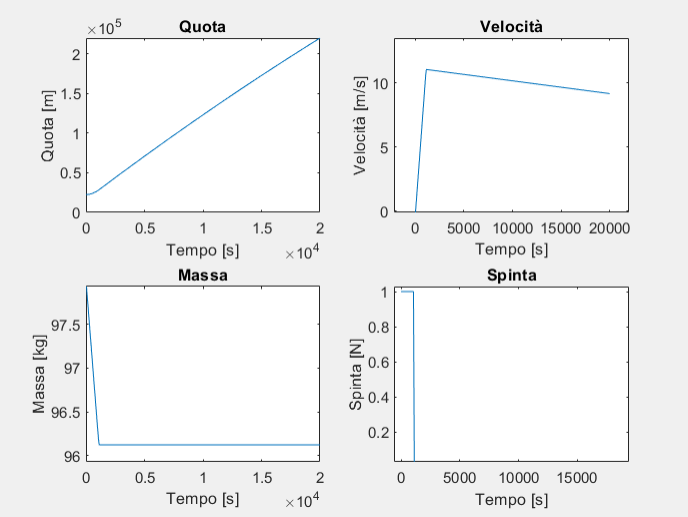


Figure 7: Risultati ottenuti con $P_1 = 1, P_2 = 1100, P_3 = 1$

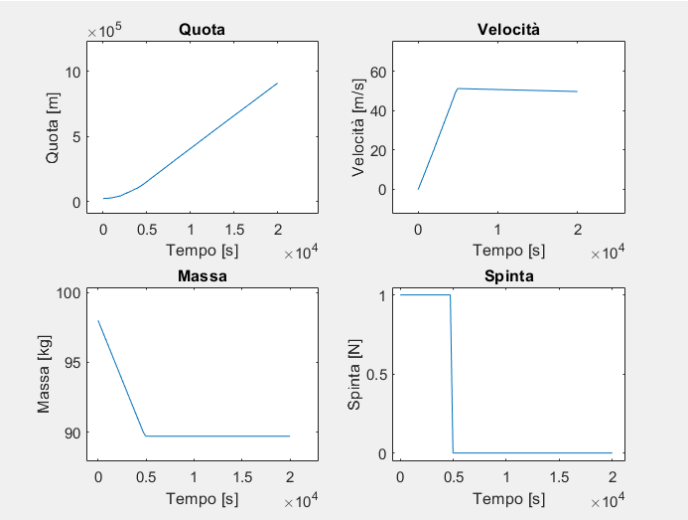


Figure 8: Risultati ottenuti con $P_1 = 1, P_2 = 5000, P_3 = 0.5$

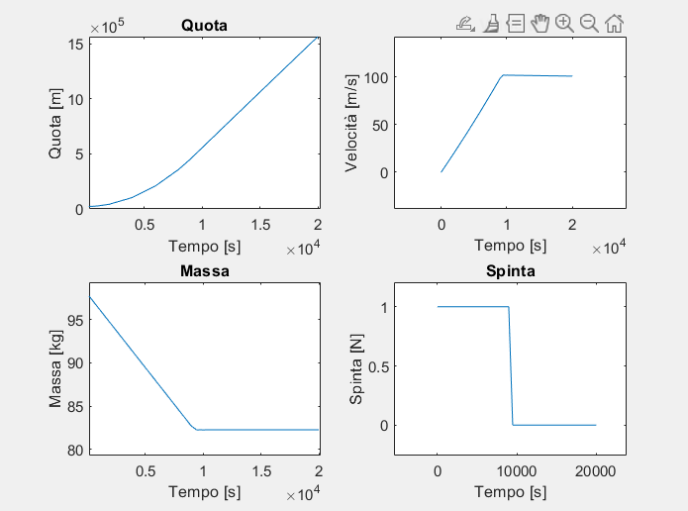


Figure 9: Risultati ottenuti con $P_1 = 1, P_2 = 10000, P_3 = 0.5$

Si può notare come a parità di P_1, aumentare la discrepanza tra P_2 e P_3 comporta un maggiore uso della spinta ed un suo spostamento dell'impulso verso destra. Per questo ho pensato di ricavare dei valori ottimali degli P_0, analizzando l'andamento di Θ: $\Theta = [\frac{P_2}{x_3} - \frac{P_3}{c}]$ inoltre da tutte le prove fatte finora ho notato che Θ ha sempre mantenuto un andamento lineare, anche se decrescente:

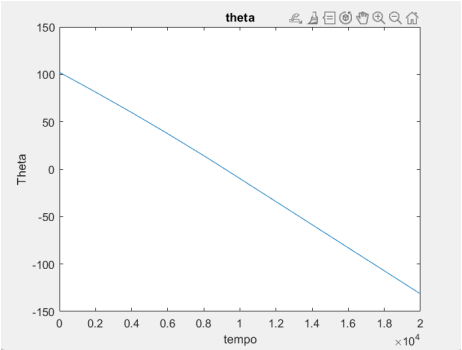


Figure 10: andamento errato di theta

Dunque per far si che il motore spinga nella fase finale della missione, bisogna impostare gli P_0 in modo tale da forzare un andamento crescente di Θ, che intersechi lo 0 (comportando l'accensione del motore) circa quando la quota è 22.75m ovvero quando t=19432.1s. Per far questo occorre analizzare l'espressione di Θ $\rightarrow \Theta = [\frac{P_2}{x_3} - \frac{P_3}{c}]$ c è una costante, e x_3, anche se può variare nel tempo, deve rimanere sempre positivo. Per le P_0 invece:

- P_2^0 dev'essere grande in modulo così che Θ sia molto negativo in t_0, e negativo perchè, dalla prima equazione di Eulero-lagrange, $\dot{P}_2 = -P_1$ e quindi la derivata di P_2 cresce se $P_1 < 0$

- $P_1^0 =$ dev'essere negativo e piccolo in modulo, così P_2 cresce lentamente, garantendo una crescita dolce di Θ

- P_3^0 dev'essere piccolo e negativo, in modo tale da minimizzare il secondo termine dell'espressione e fare in modo che Θ parta più negativa.

Impostando dei valori di P_0 di questo tipo, ci si avvicina notevolmente al risultato voluto, tuttavia anche cercando di modificare leggermente i valori dei moltiplicatori, fmin non riusciva a convergere alla soluzione deisderata:

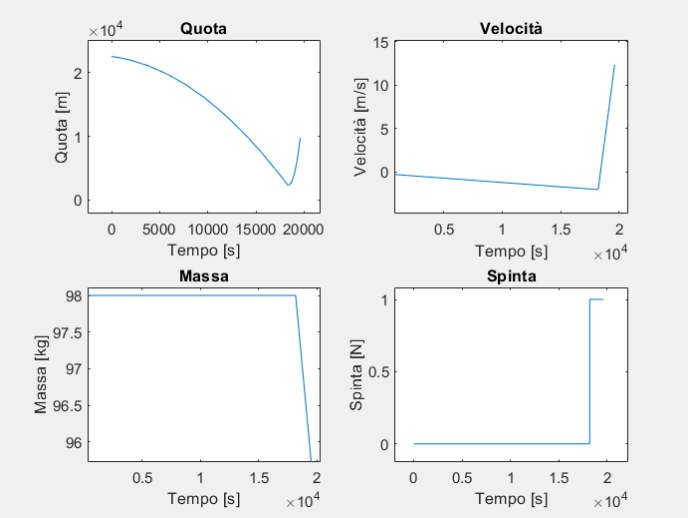


Figure 11: Run del programma con $P_1 = -0.00055, P_2 = -10, P_3 = 0.05$

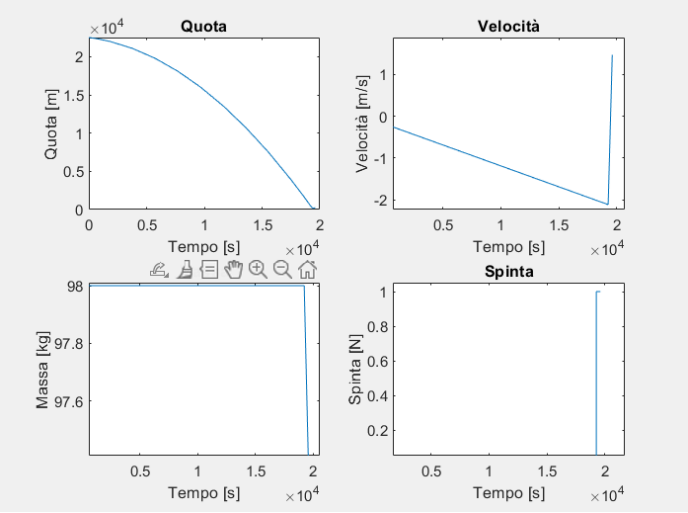


Figure 12: Run del programma con $P_1 = -0.00052, P_2 = -10, P_3 = 0.05$

Da notare l'enorme sensibilità della soluzione per una variazione di P_1 dell'ordine di 10^{-5} . Non riuscendo manualmente a trovare i valori dei moltiplicatori iniziali, ho deciso di implementare nel codice uno script che utilizzasse l'algoritmo evoluzionistico PSO, per convergere al minimo globale, conoscendo già un range in cui si potrebbe trovare il minimo globale.

7.2.1 PSO

Il PSO (Particle Swarm Optimizer) è un algoritmo che individua la zona nel dominio di ricerca delle condizioni iniziali dove si può trovare il minimo globale. Va pensato come uno sciame di vettori di stato che si muove nel dominio iterando e verificando spostamento dopo spostamento la soluzione funziome di costo. La singola particella dello sciame dunque si muove tenendo conto del suo miglior risultato, dei suoi precedenti isultati e del migliore risultato ottenuto tra tutte le particelle dello sciame. Espressione dello spostamento della singola particella appartenente allo sciame:

$$x_i^{k+1} = x_i^k + v_i^k \cdot W + c_1 \,\mathrm{rand}(1)\left[p_i^k - x_i^k\right] + c_2 \,\mathrm{rand}(1)\left[g_{\mathrm{best}}^k - x_i^k\right]$$

dove:

- W = inerzia $\rightarrow$ se è molto grande la posizione dell'iterata successiva tenderà alla direzione di quella precedente
- P_b = Personal best = miglior iterazione che la singola particella ha ottenuto in passato
- g_b^k = global best = miglior iterazione ottenuta tra tutte le particelle dello sciame
- c_1 e c_2 sono delle costanti che conferiscono una maggiore o minore importanza al termine che precedono.

In seguito riporto lo script che ho implementato per l'utilizzo del PSO:

```
clc; clear;

% === Parametri PSO ===
nPop = 1000;            % numero particelle
MaxIter = 100;          % massimo iterazioni
w = 0.7;                % inerzia
c1 = 1; c2 = 3;    % coefficienti cognitivi e sociali

nVar = 4;                               % [p1_0, p2_0, p3_0, tf]
VarSize = [1 nVar];
VarMin = [-0.00058, -9, 0.05, 19465];  % limiti inferiori
VarMax = [-0.00054, -11, 0.05, 19466];     % limiti superiori

% === Inizializzazione particelle ===
for i = 1:nPop
    % uso rand invece di unifrnd
    particle(i).Position = VarMin + (VarMax - VarMin) .* rand(1, nVar);
    particle(i).Velocity = zeros(VarSize);
    particle(i).Cost = CostFunction(particle(i).Position);
    particle(i).Best.Position = particle(i).Position;
    particle(i).Best.Cost = particle(i).Cost;
end

% Best globale iniziale
[~, idx] = min([particle.Cost]);
GlobalBest = particle(idx).Best;

% === Ciclo PSO ===
for it = 1:MaxIter
    for i = 1:nPop
        % Aggiorna velocità
        particle(i).Velocity = ...
            w*particle(i).Velocity ...
            + c1*rand(VarSize).*(particle(i).Best.Position - particle(i).Position) ...
            + c2*rand(VarSize).*(GlobalBest.Position - particle(i).Position);

        % Aggiorna posizione
        particle(i).Position = particle(i).Position + particle(i).Velocity;

```

```
        % Limita valori
        particle(i).Position = max(particle(i).Position, VarMin);
        particle(i).Position = min(particle(i).Position, VarMax);

        % Valuta costo
        particle(i).Cost = CostFunction(particle(i).Position);

        % Aggiorna best personale
        if particle(i).Cost < particle(i).Best.Cost
            particle(i).Best.Position = particle(i).Position;
            particle(i).Best.Cost = particle(i).Cost;

            % Aggiorna best globale
            if particle(i).Best.Cost < GlobalBest.Cost
                GlobalBest = particle(i).Best;
            end
        end
    end

    % Visualizza progresso
    fprintf('Iterazione %d: Costo = %.6f\n', it, GlobalBest.Cost);
end

% === Risultato finale ===
P_opt = GlobalBest.Position;
disp('Condizioni ottimali trovate:');
disp(['p1(0): ' num2str(P_opt(1))]);
disp(['p2(0): ' num2str(P_opt(2))]);
disp(['p3(0): ' num2str(P_opt(3))]);
disp(['t_f  : ' num2str(P_opt(4))]);
```

Le variabili implementare nel codice sono l'inerzia W, che ho scelto relativamente bassa, dato che conoscevo già più o meno il range dove si sarebbe potuta trovare la soluzione, il numero di particelle, che ho preso molto grande in modo tale da avere più probabilità di trovare il minimo globale, il numero di iterazioni, che ho scelto uguale a 100, dato che dopo circa 60/70 di queste, ulteriori iterazioni non comportavano ad un abbassamento dell'indice di costo ed infine il range dei P_0. Ho impostato il range di ricerca dei moltiplicatori iniziali molto ristretto, conoscendo già la zona dove il minimo ricercato si deve trovare. Il range è

$$[-0.00058, -9, 0.05, 19465] \rightarrow [-0.00054, -11, 0.05, 19466]$$

Ho scelto di mantenere P_3 fisso dato che ho notato che a parità di P_3 bastava variare i primi due per una soluzione che si avvicinava sempre di più a quella ricercata. Infine ho scelto c_1=1 e c_1=3 dando dunque un'importanza maggiore al termine dello spostamento della particella che considera il miglior risultato globale delle particelle.

8 RISULTATI

Con le precedenti premesse, il risultato che ho ottenuto lanciando lo script del PSO è:

```
        % Aggiorna best personale
        if particle(i).Cost < particle(i).Best.Cost
            particle(i).Best.Position = particle(i).Position;
            particle(i).Best.Cost = particle(i).Cost;

            % Aggiorna best globale
            if particle(i).Best.Cost < GlobalBest.Cost
                GlobalBest = particle(i).Best;
            end
        end
    end

    % Visualizza progresso
    fprintf('Iterazione %d: Costo = %.6f\n', it, GlobalBest.Cost);
end

% === Risultato finale ===
P_opt = GlobalBest.Position;
disp('Condizioni ottimali trovate:');
disp(['p1(0): ' num2str(P_opt(1))]);
disp(['p2(0): ' num2str(P_opt(2))]);
disp(['p3(0): ' num2str(P_opt(3))]);
disp(['t_f  : ' num2str(P_opt(4))]);
```

```
Command Window
  Iterazione 1: Costo = 0.833674
  Iterazione 2: Costo = 0.833674
  Iterazione 3: Costo = 0.833674
  Iterazione 4: Costo = 0.750038
  Iterazione 5: Costo = 0.750038
  Iterazione 6: Costo = 0.750038
  Iterazione 7: Costo = 0.750038
  Iterazione 8: Costo = 0.750038
  Iterazione 9: Costo = 0.750038
  Iterazione 10: Costo = 0.750038
  Iterazione 11: Costo = 0.750038
```

```
Command Window
  Iterazione 96: Costo = 0.710029
  Iterazione 97: Costo = 0.710029
  Iterazione 98: Costo = 0.710029
  Iterazione 99: Costo = 0.710029
  Iterazione 100: Costo = 0.710029
  Condizioni ottimali trovate:
  p1(0): -0.00056931
  p2(0): -11
  p3(0): 0.05
  t_f  : 19465.9349
fx >>
```

Dunque ora conosco gli P_0 in grado di trovare una funzione di costo di circa 0.7, tuttavia il PSO fa fatica da solo a convergere nel minimo, nonostante abbia efficacemente trovato la zona in cui si trova il minimo globale. A questo punto dunque non mi resta che inserire il vettore P_0 trovato dal PSO nel mio script che utilizza fmin, dato che questo è uno strumento molto più efficace per convergere precisamente al minimo. Per farlo però bisogna anche modificare i valori delle costanti β, dato che se sono troppo alti, come lo erano nel mio caso $(\beta_1, \beta_2 = 10e6)$, possono portare all'esplosione della funzione di costo:

```
    % Simulazione dinamica
    [~, X] = ode113(@(t,X) lander(t,X,T_max,g_l,c), [0 I(7)], I(1:6));

    % Estrazione variabili
    x3 = X(:,3);
    p2 = X(:,5);
    p3 = X(:,6);

    % Controllo
    theta = p2 ./ x3 - p3 / c;
    u = double(theta > 0);
    % Hamiltoniana
    H = X(:,4) .* X(:,2) + X(:,5) .* (T_max * u ./ X(:,3) - g_l) - X(:,6) .* (T_max * u / c);

    % Funzione costo (criterio di ottimalità)
    c1 = 10e6; c2 = 10e6; c3 = 1; c4 = 1;
    J = c1*abs(X(end,1) - x1_f) + ...
        c2*abs(X(end,2) - x2_f) + ...
        c3*abs(X(end,6) - 1) + ...
        c4*abs(H(end));
end
```

```
Command Window
   113          265      8.02227e+06         contract inside
   114          271      8.02227e+06         shrink
   115          272      8.02227e+06         reflect
   116          274      8.02227e+06         contract inside
   117          276      8.02227e+06         contract inside
   118          277      8.02227e+06         reflect
   119          279      8.02227e+06         contract inside
   120          281      8.02227e+06         contract inside
   121          283      8.02227e+06         contract inside
```

Questo accade perchè fmin parte dai P_0 per poi perturbarli esplorando il vicinato. Dunque per problemi come questo, dove anche minime variazioni possono portare ad una soluzione completamente diversa, dei valori alti delle β portano, ad aumentare ulteriormente la sensibilità della soluzione, sfociando nell'esplosione della funzione di costo J. Per questo ho ridimensionato i valori delle β, continuando a tener conto della maggiore importanza che hanno i termini su quota e velocità finali $\rightarrow \beta_1 = 1, \beta_2 = 1, \beta_3 = 0.001, \beta_4 = 0,001$. Dopo questo passaggio, ottengo:

```
    % Simulazione dinamica
    [~, X] = ode113(@(t,X) lander(t,X,T_max,g_l,c), [0 I(7)], I(1:6));

    % Estrazione variabili
    x3 = X(:,3);
    p2 = X(:,5);
    p3 = X(:,6);

    % Controllo
    theta = p2 ./ x3 - p3 / c;
    u = double(theta > 0);
    % Hamiltoniana
    H = X(:,4) .* X(:,2) + X(:,5) .* (T_max * u ./ X(:,3) - g_l) - X(:,6) .* (T_max * u / c);

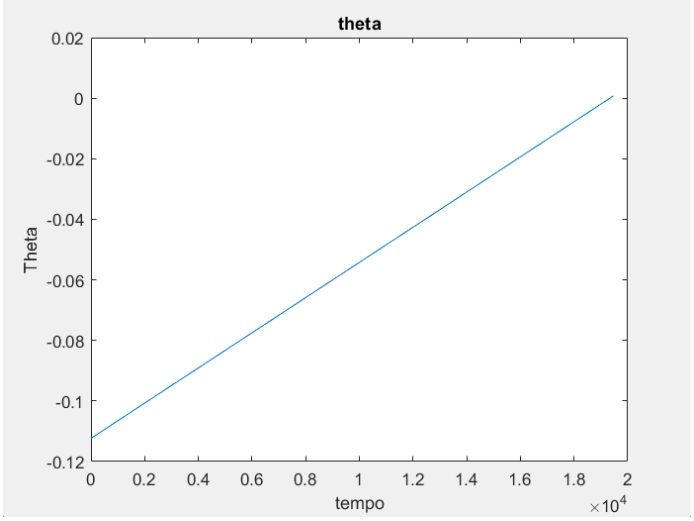
    % Funzione costo (criterio di ottimalità)
    c1 = 1; c2 = 1; c3 = 0.001; c4 = 0.001;
    J = c1*abs(X(end,1) - x1_f) + ...
        c2*abs(X(end,2) - x2_f) + ...
        c3*abs(X(end,6) - 1) + ...
        c4*abs(H(end));
end
```

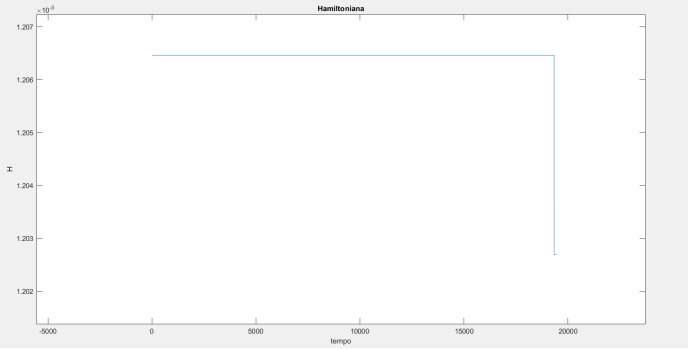
```
Command Window
   53          127         0.803211         contract inside
   54          129         0.803211         contract inside
   55          131         0.803211         contract inside

Optimization terminated:
 the current x satisfies the termination criteria using OPTIONS.TolX of 1.000000e-04
 and F(X) satisfies the convergence criteria using OPTIONS.TolFun of 1.000000e-04

   -0.0000   -0.8022
```

Dunque in realtà fmin non è riuscito a convergere ulteriormente minimizzando J, ma tuttavia i valori finali delle grandezze d'interesse trovati con il PSO non variano con quelli trovati da fmin e sono accettabili per il problema. Gli andamenti che ho voluto graficare in funzione del tempo sono: Θ, x_1, x_2, x_3, L'Hamiltoniana "H" e la spinta T, e li riporto qui sotto:





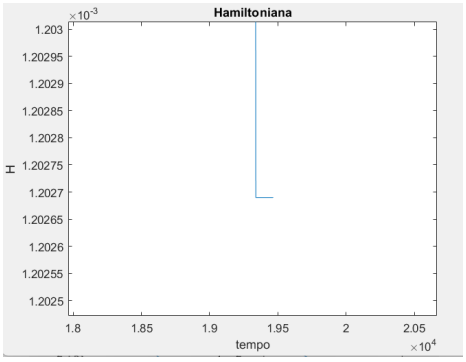
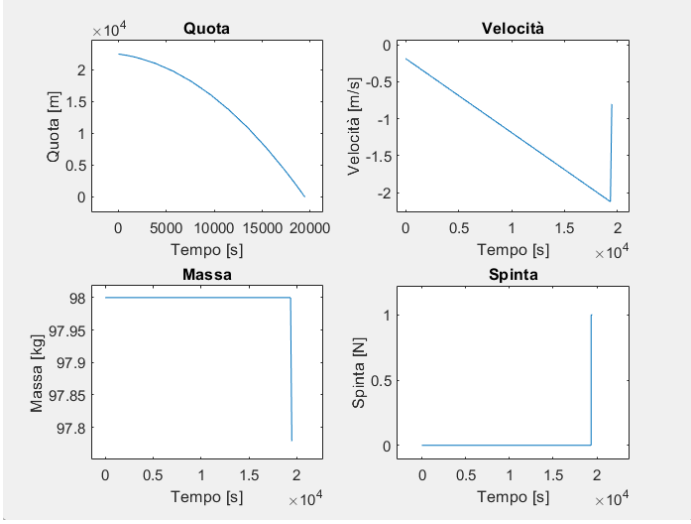


Figure 13: Tratto Finale dell'Hamiltoniana



9 Analisi dei Risultati

Dai grafici si nota come Theta ha un andamento lineare, che rimane negativo per quasi tutta la durata della discesa, per poi intersecare lo 0, in t=19336.6s, causando l'attivazione della spinta che poi rimane attiva fino al raggiungimento della quota desiderata. Di conseguenza il motore si è attivato con circa 95 secondi di anticipo rispetto alla mia previsione, ed è rimasto attivo per 129.3s.

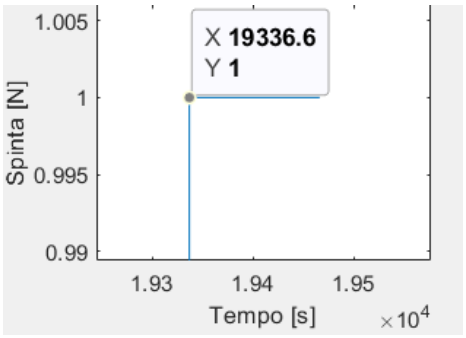


Figure 14: Zoom sulla zona d'interesse del grafico della spinta T

Dunque il lander scende in caduta libera verso la superficie della cometa sfruttando l'attrazione gravitazionale per 19336.6s, raggiungendo una velocità massima di 2.1121m/s a circa 188m dalla superficie per poi iniziare a spingere col motore, rallentando fino a raggiungere una velocità di 0.8022m/s alla quota desiderata di 0.46m in ulteriori 129.3s. La velocità finale desiderata era = 0, tuttavia l'errore sulla velocità è minimo anche considerando il fatto che i sistemi di ancoraggio del lander Philae coprono una velocità limite all'arrivo di 1.1m/s, al di sopra di quella prevista dalla simulazione. Riporto nella seguente tabella i valori finali delle grandezze riportate ed i relativi errori (dove necessario):

Grandezza	Valore Finale	Errore
Quota(m)	0,4600339	-3.385943613293252e-05
Velocità(m/s)	0.8022	0.8022
tempo(s)	19465.9349000000	
massa(kg)	97.7785823254515	
Hamiltoniana	0.00120269	0.00120269

Table 1: VALORI FINALI

Da come si nota nel grafico, l'Hamiltoniana è costante nel primo tratto della traiettoria, quando il lander è in caduta libera verso la cometa. Appena si attiva la spinta, l'Hamiltoniana subisce una discontinuità dovuta al fatto che u passa da 0 a 1 e dunque nasce un nuovo contributo nell'espressione dell'Hamiltoniana, che inevitabilmente cambia valore. Tuttavia si tratta di un salto istantaneo e dell'ordine di 10^{-6}, che ci permette di avere un Hamiltoniana che continua ad essere costante anche nel tratto finale e prossima a 0, come ci si aspettava. Infine dato che l'obiettivo primario del problema era quello di massimizzare la massa, sono stati impiegati durante la discesa solamente 0.221 kg di propellente. Di seguito riporto uno zoom effettuato sul grafico della massa in funzione del tempo, nella fase finale, dove effettivamente la massa diminuisce, così da rendere l'andamento lineare decrescente riconoscibile.

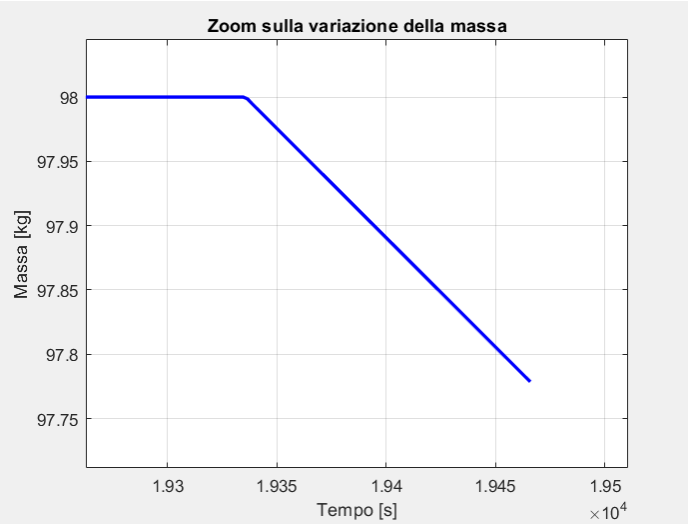


Figure 15: Andamento della massa nell'intervallo 19330s-19470s